

Exercise 1: Implementing the Singleton Pattern

Scenario: You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Objective: The objective of this exercise is to implement the Singleton design pattern in Java to ensure only one instance of a class is created and shared across the application.

1. Understanding Singleton Pattern

The Singleton Pattern is a design pattern that restricts the instantiation of a class to one single object.

It is used when:

- Only one object is needed globally
- Shared resources like logging, configuration, database connection are required
- Consistency is important across the application

Key Points:

- Private constructor prevents object creation from outside
- Static instance ensures single object
- Public method provides global access point

2. Setup

A Logger class is created with a private static instance and a private constructor to prevent multiple object creation.

3. Implementation

Singleton Class (Logger)

```
class Logger {  
  
    private static Logger instance;  
  
    private Logger() {  
    }  
  
    public static Logger getInstance() {  
        if(instance == null) {  
            instance = new Logger();  
        }  
        return instance;  
    }  
  
    public void log(String message) {  
        System.out.println("Log: " + message);  
    }  
}
```

4. Testing Singleton Implementation

5. Output Screenshot

```
Log: First message
Log: Second message
true
```



Exercise 2: Implementing the Factory Method Pattern

Objective: The objective is to implement the Factory Method design pattern to create objects without exposing the instantiation logic to the client.

1. Understanding Factory Method Pattern

The Factory Method Pattern defines an interface or abstract class for creating objects but allows subclasses to decide which class to instantiate.

It is used when:

- Object creation logic should be centralized
- System needs flexibility to add new types easily
- Client should not know exact class being created

2. Setup

A Document interface is created for different document types.

```
interface Document {  
    void open();  
}
```

3. Concrete Classes

```
class WordDocument implements Document {  
    public void open() {  
        System.out.println("Opening Word Document");  
    }  
}
```

```
class PdfDocument implements Document {  
    public void open() {  
        System.out.println("Opening PDF Document");  
    }  
}
```

```
class ExcelDocument implements Document {  
    public void open() {  
        System.out.println("Opening Excel Document");  
    }  
}
```

4. Factory Method Implementation

```
abstract class DocumentFactory {  
    abstract Document createDocument();  
}
```

```
class WordFactory extends DocumentFactory {  
    public Document createDocument() {  
        return new WordDocument();  
    }  
}
```

```
class PdfFactory extends DocumentFactory {  
    public Document createDocument() {
```

```
return new PdfDocument();  
}  
}
```

```
class ExcelFactory extends DocumentFactory {  
    public Document createDocument() {  
        return new ExcelDocument();  
    }  
}
```

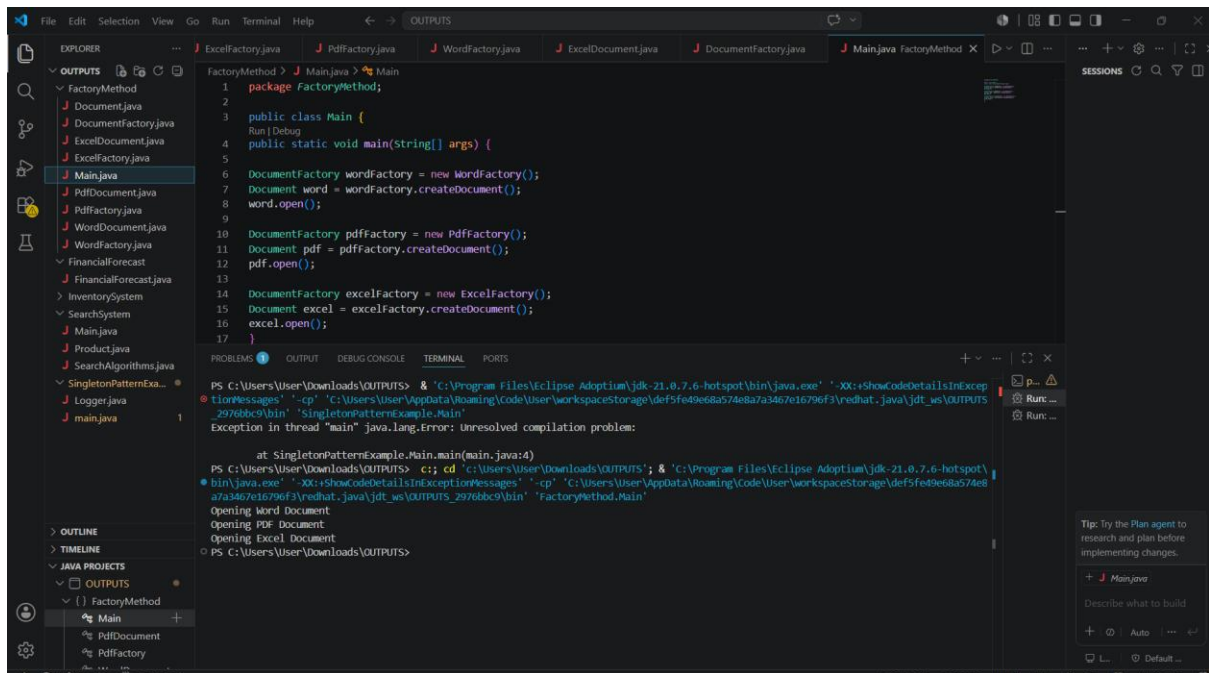
5. Testing Factory Method Implementation

```
public class Main {  
  
    public static void main(String[] args) {  
  
        DocumentFactory wordFactory = new WordFactory();  
        Document word = wordFactory.createDocument();  
        word.open();  
  
        DocumentFactory pdfFactory = new PdfFactory();  
        Document pdf = pdfFactory.createDocument();  
        pdf.open();  
  
        DocumentFactory excelFactory = new ExcelFactory();  
        Document excel = excelFactory.createDocument();  
        excel.open();  
    }  
}
```

6. Output Screenshot

Expected Output:

Opening Word Document
Opening PDF Document
Opening Excel Document



7.Conclusion

The Factory Method Pattern provides a flexible way to create objects without exposing instantiation logic. It improves scalability and makes the system easy to extend with new document types.